Advanced Rust (2026): Unsafe Rust

Lecture 5

Lukáš Hozda

Spring 2026

MFF CUNI

Unsafe Rust

Safe Rust Guarantees

Safe Rust makes the following set of promises, it forbids/prevents:

Safe Rust Guarantees

Safe Rust makes the following set of promises, it forbids/prevents:

- dereferencing dangling or unaligned pointers

Safe Rust Guarantees

Safe Rust makes the following set of promises, it forbids/prevents:

- dereferencing dangling or unaligned pointers
- observing uninitialized memory through safe APIs

Safe Rust Guarantees

Safe Rust makes the following set of promises, it forbids/prevents:

- dereferencing dangling or unaligned pointers
- observing uninitialized memory through safe APIs
- mutable aliasing violations

Safe Rust Guarantees

Safe Rust makes the following set of promises, it forbids/prevents:

- dereferencing dangling or unaligned pointers
- observing uninitialized memory through safe APIs
- mutable aliasing violations
- data races

Safe Rust Guarantees

Safe Rust makes the following set of promises, it forbids/prevents:

- dereferencing dangling or unaligned pointers
- observing uninitialized memory through safe APIs
- mutable aliasing violations
- data races
- producing values that violate type invariants

Safe Rust Guarantees

Safe Rust makes the following set of promises, it forbids/prevents:

- dereferencing dangling or unaligned pointers
- observing uninitialized memory through safe APIs
- mutable aliasing violations
- data races
- producing values that violate type invariants

Rust does **not** prevent memory leaks, deadlocks, logic bugs, `Rc` cycles, and infinite loops, though.

`unsafe` Contract

`unsafe` more of a contract and not “turn off the rules.”

`unsafe` Contract

`unsafe` more of a contract and not “turn off the rules.”

We tell the compiler: “I have verified properties you cannot check. Hold me to everything else.”

`unsafe` Contract

`unsafe` more of a contract and not “turn off the rules.”

We tell the compiler: “I have verified properties you cannot check. Hold me to everything else.”

Type system, the borrow checker, lifetime rules are all still active inside `unsafe`.

`unsafe` Contract

`unsafe` more of a contract and not “turn off the rules.”

We tell the compiler: “I have verified properties you cannot check. Hold me to everything else.”

Type system, the borrow checker, lifetime rules are all still active inside `unsafe`.

We gain access to five specific operations the compiler cannot verify on its own, however

Five Unsafe Operations

1. Dereferencing raw pointers
2. Calling `unsafe fn`
3. Accessing or mutating `static mut`
4. Implementing `unsafe trait`
5. Reading union fields

`unsafe` keyword places

- `unsafe fn`: the **caller** must satisfy extra preconditions

`unsafe` keyword places

- `unsafe fn`: the **caller** must satisfy extra preconditions
- `unsafe { ... }`: the author asserts specific operations are sound here

`unsafe` keyword places

- `unsafe fn`: the **caller** must satisfy extra preconditions
- `unsafe { ... }`: the author asserts specific operations are sound here
- `unsafe trait`: implementors promise semantic guarantees the compiler cannot check

`unsafe` keyword places

- `unsafe fn`: the **caller** must satisfy extra preconditions
- `unsafe { ... }`: the author asserts specific operations are sound here
- `unsafe trait`: implementors promise semantic guarantees the compiler cannot check
- `unsafe impl`: you claim those guarantees hold for your type

`unsafe` keyword places

- `unsafe fn`: the **caller** must satisfy extra preconditions
- `unsafe { ... }`: the author asserts specific operations are sound here
- `unsafe trait`: implementors promise semantic guarantees the compiler cannot check
- `unsafe impl`: you claim those guarantees hold for your type

`unsafe fn` pushes proof obligations outward; an `unsafe` block absorbs them inward.

Attitude

Gjengset: most Rust programmers either treat unsafe with fear or dismiss it as “just C.”

Attitude

Gjengset: most Rust programmers either treat unsafe with fear or dismiss it as “just C.”

Both notions are incorrect.

Attitude

Gjengset: most Rust programmers either treat unsafe with fear or dismiss it as “just C.”

Both notions are incorrect.

- Unsafe Rust can be **harder** than C

Attitude

Gjengset: most Rust programmers either treat unsafe with fear or dismiss it as “just C.”

Both notions are incorrect.

- Unsafe Rust can be **harder** than C
- The C compiler makes few promises, so it assumes little

Attitude

Gjengset: most Rust programmers either treat unsafe with fear or dismiss it as "just C."

Both notions are incorrect.

- Unsafe Rust can be **harder** than C
- The C compiler makes few promises, so it assumes little
- In Rust, the optimizer relies on strong guarantees

Attitude

Gjengset: most Rust programmers either treat unsafe with fear or dismiss it as "just C."

Both notions are incorrect.

- Unsafe Rust can be **harder** than C
- The C compiler makes few promises, so it assumes little
- In Rust, the optimizer relies on strong guarantees
  - `unsafe` code must uphold all of them, even though the compiler cannot check

Unsafe usecases

Hardware

```
const MMIO_REG: *mut u32 = 0x1000_0000 as *mut u32;

unsafe fn poke(value: u32) {
    std::ptr::write_volatile(MMIO_REG, value);
}
```

Hardware

```
const MMIO_REG: *mut u32 = 0x1000_0000 as *mut u32;

unsafe fn poke(value: u32) {
    std::ptr::write_volatile(MMIO_REG, value);
}
```

- The compiler is not psychic, it has no model of your memory map.

Hardware

```
const MMIO_REG: *mut u32 = 0x1000_0000 as *mut u32;

unsafe fn poke(value: u32) {
    std::ptr::write_volatile(MMIO_REG, value);
}
```

- The compiler is not psychic, it has no model of your memory map.
- It cannot know whether `0x1000_0000` is a valid device register.

Hardware

```
const MMIO_REG: *mut u32 = 0x1000_0000 as *mut u32;

unsafe fn poke(value: u32) {
    std::ptr::write_volatile(MMIO_REG, value);
}
```

- The compiler is not psychic, it has no model of your memory map.
- It cannot know whether `0x1000_0000` is a valid device register.

HAL crates hide this from application code. But somebody has to write the HAL.

Example numero dos: VGA Buffer

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

Example numero dos: VGA Buffer

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Safety depends entirely on facts outside the program

Example numero dos: VGA Buffer

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Safety depends entirely on facts outside the program
  - is this address mapped?
  - Is there an OS managing it?
  - Are we the only accessor?

Example numero dos: VGA Buffer

```
const VGA_PTR: *mut u8 = 0x8080_8080 as *mut u8;

let vga = unsafe {
    std::slice::from_raw_parts_mut(VGA_PTR, 800 * 600)
};
```

- Safety depends entirely on facts outside the program
  - is this address mapped?
  - Is there an OS managing it?
  - Are we the only accessor?

E.g. On baremetal before boot, this is fine. In a userspace process, it's nonsense

FFI

```
extern "C" {
    fn strcpy(
        dest: *mut libc::c_char,
        src: *const libc::c_char,
    ) -> *mut libc::c_char;
}
```

```
extern "C" {
    fn strcpy(
        dest: *mut libc::c_char,
        src: *const libc::c_char,
    ) -> *mut libc::c_char;
}
```

The Rust compiler is still not psychic and cannot inspect the implementation of `strcpy`. It doesnt know that `strcpy` requires a null terminator, that it writes past `dest` if the buffer is too small, or that the source and destination must not overlap.

```
extern "C" {
    fn strcpy(
        dest: *mut libc::c_char,
        src: *const libc::c_char,
    ) -> *mut libc::c_char;
}
```

The Rust compiler is still not psychic and cannot inspect the implementation of `strcpy`. It doesnt know that `strcpy` requires a null terminator, that it writes past `dest` if the buffer is too small, or that the source and destination must not overlap.

Rustc sees is a function signature and a promise that you will call it correctly

Safe Wrappers

```
fn safe_strcpy(dest: &mut [u8], src: &str) -> Result<(), &'static str> {
    if dest.len() <= src.len() {
        return Err("destination buffer too small");
    }
    let c_src = std::ffi::CString::new(src)
        .expect("interior null byte");
    unsafe {
        strcpy(
            dest.as_mut_ptr() as *mut libc::c_char,
            c_src.as_ptr(),
        );
    }
    Ok(())
}
```

We minimize the unsafe part. The safe wrapper encodes the preconditions as types and checks

Concurrency Primitives

`Mutex<T>` hands out `&mut T` through `&self`.

Concurrency Primitives

`Mutex<T>` hands out `&mut T` through `&self`.

That looks like it violates the aliasing rule: you have `&Mutex<T>` (shared) and produce `&mut T` (exclusive).

Concurrency Primitives

`Mutex<T>` hands out `&mut T` through `&self`.

That looks like it violates the aliasing rule: you have `&Mutex<T>` (shared) and produce `&mut T` (exclusive).

We use `UnsafeCell<T>`, the **only** legitimate way to get `&mut` from `&` in Rust. All interior-mutability types (`Cell`, `RefCell`, `Mutex`, `RwLock`, `AtomicU64`) use it.

Concurrency Primitives

`Mutex<T>` hands out `&mut T` through `&self`.

That looks like it violates the aliasing rule: you have `&Mutex<T>` (shared) and produce `&mut T` (exclusive).

We use `UnsafeCell<T>`, the **only** legitimate way to get `&mut` from `&` in Rust. All interior-mutability types (`Cell`, `RefCell`, `Mutex`, `RwLock`, `AtomicU64`) use it.

- The lock protocol ensures exclusivity at runtime.
- The `unsafe` block is where you assert that the protocol works.

Borrow Checker not being smart enough

```
fn next(buffer: &mut String) -> &str {
    loop {
        let event = parse(buffer);
        if true { return event; }
    }
}
fn parse(buffer: &mut String) -> &str { unimplemented!() }
```

Borrow Checker not being smart enough

```
fn next(buffer: &mut String) -> &str {
    loop {
        let event = parse(buffer);
        if true { return event; }
    }
}
fn parse(buffer: &mut String) -> &str { unimplemented!() }
```

This is correct, but the borrow checker rejects it because it cannot see that the early return ends the mutable borrow.

Borrow Checker not being smart enough

```
fn next(buffer: &mut String) -> &str {
    loop {
        let event = parse(buffer);
        if true { return event; }
    }
}
fn parse(buffer: &mut String) -> &str { unimplemented!() }
```

This is correct, but the borrow checker rejects it because it cannot see that the early return ends the mutable borrow.

Performance

```rust
fn sum_safe(v: &[i32]) -> i32 {
    v.iter().copied().sum()
}

fn sum_raw(v: &[i32]) -> i32 {
    let (mut sum, ptr, len) = (0, v.as_ptr(), v.len());
    for i in 0..len {
        sum += unsafe { *ptr.add(i) };
    }
    sum
}
```

Performance

```rust
fn sum_safe(v: &[i32]) -> i32 {
    v.iter().copied().sum()
}

fn sum_raw(v: &[i32]) -> i32 {
    let (mut sum, ptr, len) = (0, v.as_ptr(), v.len());
    for i in 0..len {
        sum += unsafe { *ptr.add(i) };
    }
    sum
}
```

In this case, LLVM already eliminates the bounds check in `sum_safe`. The unsafe version buys nothing and costs an ongoing proof obligation.

Performance

```rust
fn sum_safe(v: &[i32]) -> i32 {
    v.iter().copied().sum()
}

fn sum_raw(v: &[i32]) -> i32 {
    let (mut sum, ptr, len) = (0, v.as_ptr(), v.len());
    for i in 0..len {
        sum += unsafe { *ptr.add(i) };
    }
    sum
}
```

In this case, LLVM already eliminates the bounds check in `sum_safe`. The unsafe version buys nothing and costs an ongoing proof obligation.

Undefined Behavior

Types as Invariants

A type is a set of valid values:

Types as Invariants

A type is a set of valid values:

- `bool`: only `0` and `1`, not `2` or eg `0xFF`

Types as Invariants

A type is a set of valid values:

- `bool`: only `0` and `1`, not `2` or eg `0xFF`
  - For instance, `Option<bool>` depends on this to encode into one byte

Types as Invariants

A type is a set of valid values:

- `bool`: only `0` and `1`, not `2` or eg `0xFF`
  - For instance, `Option<bool>` depends on this to encode into one byte
- `char`: only valid Unicode scalar values

Types as Invariants

A type is a set of valid values:

- `bool`: only `0` and `1`, not `2` or eg `0xFF`
  - For instance, `Option<bool>` depends on this to encode into one byte
- `char`: only valid Unicode scalar values
- `!`: **no** values at all

Types as Invariants

A type is a set of valid values:

- `bool`: only `0` and `1`, not `2` or eg `0xFF`
  - For instance, `Option<bool>` depends on this to encode into one byte
- `char`: only valid Unicode scalar values
- `!`: **no** values at all
- `&T`: non-null, aligned, points to a valid `T`, no coexisting `&mut T`

Types as Invariants

A type is a set of valid values:

- `bool`: only `0` and `1`, not `2` or eg `0xFF`
  - For instance, `Option<bool>` depends on this to encode into one byte
- `char`: only valid Unicode scalar values
- `!`: **no** values at all
- `&T`: non-null, aligned, points to a valid `T`, no coexisting `&mut T`

Producing a value outside these sets is UB the moment it exists

Compiler Assumptions

The optimizer does not treat `unsafe` blocks specially. It assumes **all** code respects Rust's rules:

Compiler Assumptions

The optimizer does not treat `unsafe` blocks specially. It assumes **all** code respects Rust's rules:

- References are non-null, aligned, dereferenceable, and point to valid data
- `&mut` is exclusive, `&` is shared, and the two never coexist for the same location
- Values match their type (a `bool` is 0 or 1, a `char` is a valid Unicode scalar, `!` has no values)
- No data races occur

Compiler Assumptions

The optimizer does not treat `unsafe` blocks specially. It assumes **all** code respects Rust's rules:

- References are non-null, aligned, dereferenceable, and point to valid data
- `&mut` is exclusive, `&` is shared, and the two never coexist for the same location
- Values match their type (a `bool` is 0 or 1, a `char` is a valid Unicode scalar, `!` has no values)
- No data races occur

If any of these are violated, even inside `unsafe`, the optimizer is no longer constrained.

Invalid Values

```
let raw: u8 = 2;
let flag: bool = unsafe { std::mem::transmute(raw) };
```

Invalid Values

```
let raw: u8 = 2;
let flag: bool = unsafe { std::mem::transmute(raw) };
```

This is UB already at the `transmute`. The `bool` type has exactly two inhabitants: `0` and `1`. Producing a `bool` with value `2` is already a lie to the compiler.

Invalid Values

```
let raw: u8 = 2;
let flag: bool = unsafe { std::mem::transmute(raw) };
```

This is UB already at the `transmute`. The `bool` type has exactly two inhabitants: `0` and `1`. Producing a `bool` with value `2` is already a lie to the compiler.

Never Type

```
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

Never Type

```rust
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

`!` has zero inhabitants. Producing one is instant UB. The compiler may assume this function never returns. And if it does, anything goes.

Never Type

```rust
fn paradox() -> ! {
    unsafe { std::mem::transmute(()) }
}
```

`!` has zero inhabitants. Producing one is instant UB. The compiler may assume this function never returns. And if it does, anything goes.

Aliasing

Consider this function:

```
fn compute(i: &u32, o: &mut u32) {
    if *i > 10 { *o = 1; }
    if *i > 5  { *o *= 2; }
}
```

Aliasing

The compiler can transform it to:

```
fn compute(i: &u32, o: &mut u32) {
    let cached = *i;
    if cached > 10 { *o = 2; }
    else if cached > 5 { *o *= 2; }
}
```

Aliasing

The compiler can transform it to:

```
fn compute(i: &u32, o: &mut u32) {
    let cached = *i;
    if cached > 10 { *o = 2; }
    else if cached > 5 { *o *= 2; }
}
```

- This is legal because `&u32` and `&mut u32` cannot alias in Rust, so they cannot point to the same thing.

Aliasing

The compiler can transform it to:

```
fn compute(i: &u32, o: &mut u32) {
    let cached = *i;
    if cached > 10 { *o = 2; }
    else if cached > 5 { *o *= 2; }
}
```

- This is legal because `&u32` and `&mut u32` cannot alias in Rust, so they cannot point to the same thing.
- The compiler is allowed to cache `*i` across the write to `*o`.

Aliasing

The compiler can transform it to:

```
fn compute(i: &u32, o: &mut u32) {
    let cached = *i;
    if cached > 10 { *o = 2; }
    else if cached > 5 { *o *= 2; }
}
```

- This is legal because `&u32` and `&mut u32` cannot alias in Rust, so they cannot point to the same thing.
- The compiler is allowed to cache `*i` across the write to `*o`.

In C, this optimization is **not** legal. Because `i` and `o` could point to the same `int`. Stricter default semantics can lead to better codegen.

Aliasing

The compiler can transform it to:

```
fn compute(i: &u32, o: &mut u32) {
    let cached = *i;
    if cached > 10 { *o = 2; }
    else if cached > 5 { *o *= 2; }
}
```

- This is legal because `&u32` and `&mut u32` cannot alias in Rust, so they cannot point to the same thing.
- The compiler is allowed to cache `*i` across the write to `*o`.

In C, this optimization is **not** legal. Because `i` and `o` could point to the same `int`. Stricter default semantics can lead to better codegen.

If your unsafe code violate this, the optimizer may produce wrong code.

Reference Invariants

```
let ptr = 0x1usize as *const u8;
let r: &u8 = unsafe { &*ptr };
```

Reference Invariants

```
let ptr = 0x1usize as *const u8;
let r: &u8 = unsafe { &*ptr };
```

Forming the reference, not dereferencing it, is already the dangerous part. A `&u8` promises:

- Non-null
- Properly aligned
- Points to initialized, valid `u8`
- The referent is live for the reference's lifetime
- No `&mut u8` exists to this location simultaneously

Packed Structs

```
#[repr(C, packed)]
struct Header { tag: u8, len: u16 }

let h = Header { tag: 1, len: 0x1234 };
// let r = &h.len; // UB: creates unaligned reference
let len = unsafe {
    std::ptr::addr_of!(h.len).read_unaligned()
};
```

Packed Structs

```
#[repr(C, packed)]
struct Header { tag: u8, len: u16 }

let h = Header { tag: 1, len: 0x1234 };
// let r = &h.len; // UB: creates unaligned reference
let len = unsafe {
    std::ptr::addr_of!(h.len).read_unaligned()
};
```

`packed` removes padding, field accesses can become alignment issues.

Raw Memory

Manual Ownership

```
struct BufferHolder { buf: *mut i32, len: usize }

impl BufferHolder {
    fn new(n: usize) -> Self {
        let mut v = vec![0i32; n];
        let ptr = v.as_mut_ptr();
        std::mem::forget(v);      // ownership transfers to us
        Self { buf: ptr, len: n }
    }
}
```

Manual Ownership

```
struct BufferHolder { buf: *mut i32, len: usize }

impl BufferHolder {
    fn new(n: usize) -> Self {
        let mut v = vec![0i32; n];
        let ptr = v.as_mut_ptr();
        std::mem::forget(v);      // ownership transfers to us
        Self { buf: ptr, len: n }
    }
}
```

`mem::forget` prevents `Vec`'s destructor from running. The allocation is now our problem: we must free it exactly once.

```
impl Drop for BufferHolder {
    fn drop(&mut self) {
        unsafe {
            // reconstruct the Vec so its Drop frees the allocation.
            // len == capacity because we allocated with vec![0; n].
            drop(Vec::from_raw_parts(self.buf, self.len, self.len));
        }
    }
}
```

```
impl Drop for BufferHolder {
    fn drop(&mut self) {
        unsafe {
            // reconstruct the Vec so its Drop frees the allocation.
            // len == capacity because we allocated with vec![0; n].
            drop(Vec::from_raw_parts(self.buf, self.len, self.len));
        }
    }
}
```

If we get `len` or capacity wrong, or call this twice, or forget to implement `Drop` at all: double-free, use-after-free, or leak.

MaybeUninit

Uninitialized memory is not “a `T` with unknown contents.” It is **not yet a `T`**.

MaybeUninit

Uninitialized memory is not "a `T` with unknown contents." It is **not yet a `T`**.

```
use std::mem::MaybeUninit;

let mut slot = MaybeUninit::<u32>::uninit();
slot.write(42);
let val = unsafe { slot.assume_init() }; // now it's a u32
```

MaybeUninit

Uninitialized memory is not "a `T` with unknown contents." It is **not yet a `T`**.

```
use std::mem::MaybeUninit;

let mut slot = MaybeUninit::<u32>::uninit();
slot.write(42);
let val = unsafe { slot.assume_init() }; // now it's a u32
```

`assume_init()` is the exact point where you assert the value is valid. Before that line, reading it as `u32` would be UB.

Panic Safety

```
impl<T> Vec<T> {
    fn extend_map<U, F: FnMut(&U) -> T>(
        &mut self, src: &[U], mut f: F,
    ) {
        self.reserve(src.len());
        unsafe { self.set_len(self.len() + src.len()); } // <-- !
        let mut dst = unsafe { self.as_mut_ptr().add(self.len() - src.len()) };
        for u in src {
            unsafe { std::ptr::write(dst, f(u)); }
            dst = unsafe { dst.add(1) };
        }
    }
}
```

Panic Safety

- The `set_len` call tells `Vec` that all new slots are initialized **before** `f` has run.

Panic Safety

- The `set_len` call tells `Vec` that all new slots are initialized **before** `f` has run.
- If `f` panics on the third element, unwinding calls `drop()` on the `Vec`
- Destructor drops all `self.len()` elements (including slots that were never written to :()

Panic Safety

- The `set_len` call tells `Vec` that all new slots are initialized **before** `f` has run.
- If `f` panics on the third element, unwinding calls `drop()` on the `Vec`
- Destructor drops all `self.len()` elements (including slots that were never written to :()
- Dropping uninitialized memory is obviously UB
- The bug doesn't show on the happy path and only surfaces when (even unrelated) code panics

Panic Safety

- The `set_len` call tells `Vec` that all new slots are initialized **before** `f` has run.
- If `f` panics on the third element, unwinding calls `drop()` on the `Vec`
- Destructor drops all `self.len()` elements (including slots that were never written to :()
- Dropping uninitialized memory is obviously UB
- The bug doesn't show on the happy path and only surfaces when (even unrelated) code panics

The fix: increment the length **after** each successful write, not before the loop. (Or use a drop guard that resets the length on panic.)

Safe Abstractions

Caller Obligations

If your function signature has no `unsafe`, every caller is entitled to assume it cannot cause UB, and they can

Caller Obligations

If your function signature has no `unsafe`, every caller is entitled to assume it cannot cause UB, and they can

- can call it with any value the type permits
- can panic in callbacks
- can clone, drop, or leak the result
- can do all of this from multiple threads if `Send`/`Sync` allow it

Caller Obligations

If your function signature has no `unsafe`, every caller is entitled to assume it cannot cause UB, and they can

- can call it with any value the type permits
- can panic in callbacks
- can clone, drop, or leak the result
- can do all of this from multiple threads if `Send`/`Sync` allow it

Module-Wide Proof Surface

We cannot be auditing only the `unsafe { }` blocks.

Module-Wide Proof Surface

We cannot be auditing only the `unsafe { }` blocks.

Safe code in the **same module** can invalidate the invariants that unsafe code depends on. Constructors, setters, `Clone`, `Drop`, and anything that touches the private fields are all part of the proof.

Module-Wide Proof Surface

We cannot be auditing only the `unsafe { }` blocks.

Safe code in the **same module** can invalidate the invariants that unsafe code depends on. Constructors, setters, `Clone`, `Drop`, and anything that touches the private fields are all part of the proof.

```
mod my_vec {
    pub struct MyVec<T> { ptr: *mut T, len: usize, cap: usize }

    // this safe fn can break the unsafe code in `push` and `drop`
    // if it sets len > cap or len > initialized count.
    pub fn corrupt(&mut self) { self.len = 999; }
}
```

Unsafe Traits

```
unsafe trait GlobalInvariant {
    // Implementors promise some property the compiler cannot check.
}

unsafe impl GlobalInvariant for MyType {
    // "I assert that MyType upholds the invariant."
}
```

Unsafe Traits

```
unsafe trait GlobalInvariant {
    // Implementors promise some property the compiler cannot check.
}

unsafe impl GlobalInvariant for MyType {
    // "I assert that MyType upholds the invariant."
}
```

`Send` and `Sync` are the canonical examples. The promise is not local, which means the consequences are not local either.

Minimizing Unsafe Surface

- Small `unsafe` blocks are easier to audit

Minimizing Unsafe Surface

- Small `unsafe` blocks are easier to audit
- Extract raw operations into a safe local helper, then use the helper

Minimizing Unsafe Surface

- Small `unsafe` blocks are easier to audit
- Extract raw operations into a safe local helper, then use the helper

Tooling

Miri

Miri interprets Rust at the MIR level and checks safety properties at runtime: use-after-free, out-of-bounds access, uninitialized reads, aliasing violations (Stacked Borrows / Tree Borrows).

Miri

Miri interprets Rust at the MIR level and checks safety properties at runtime: use-after-free, out-of-bounds access, uninitialized reads, aliasing violations (Stacked Borrows / Tree Borrows).

Miri

Miri interprets Rust at the MIR level and checks safety properties at runtime: use-after-free, out-of-bounds access, uninitialized reads, aliasing violations (Stacked Borrows / Tree Borrows).

Miri is not a proof of correctness, but it catches classes of bugs the compiler usually doesn't.

Miri

```
struct Oops(Vec<u8>);
impl Oops {
    fn new() -> Self {
        let v = Vec::with_capacity(10);
        let (ptr, _, cap) = v.into_raw_parts();
        // Claims 10 elements exist, but nothing was written.
        let v = unsafe { Vec::from_raw_parts(ptr, 10, cap) };
        Self(v)
    }
}
fn main() {
    println!("{}", Oops::new().0[0]); // reads uninit memory
}
```

Miri

```
struct Oops(Vec<u8>);
impl Oops {
    fn new() -> Self {
        let v = Vec::with_capacity(10);
        let (ptr, _, cap) = v.into_raw_parts();
        // Claims 10 elements exist, but nothing was written.
        let v = unsafe { Vec::from_raw_parts(ptr, 10, cap) };
        Self(v)
    }
}
fn main() {
    println!("{}", Oops::new().0[0]); // reads uninit memory
}
```

Miri will flag this

Sanitizers

- **AddressSanitizer**: catches out-of-bounds heap/stack access, use-after-free, double-free (works with `RUSTFLAGS="-Zsanitizer=address"`)

Sanitizers

- **AddressSanitizer**: catches out-of-bounds heap/stack access, use-after-free, double-free (works with `RUSTFLAGS="-Zsanitizer=address"`)
- **ThreadSanitizer**: catches data races across threads

Sanitizers

- **AddressSanitizer**: catches out-of-bounds heap/stack access, use-after-free, double-free (works with `RUSTFLAGS="-Zsanitizer=address"`)
- **ThreadSanitizer**: catches data races across threads
- **cargo-careful**: runs your code with extra debug assertions from the standard library enabled

Sanitizers

- **AddressSanitizer**: catches out-of-bounds heap/stack access, use-after-free, double-free (works with `RUSTFLAGS="-Zsanitizer=address"`)
- **ThreadSanitizer**: catches data races across threads
- **cargo-careful**: runs your code with extra debug assertions from the standard library enabled

None of these **prove** correctness, these belong in CI along with other checks.